



Safety Verification of Chiron IR using Constrained Horn Clauses

Aditi Khandelia Arush Upadhyaya Kushagra Srivastava

Introduction

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a Property Directed Reachability (PDR)-based safety verification engine, that given a program and a set of properties on the program state, either guarantees that they hold or provides a counterexample for the same. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Base Flow & Tools Used

Overview of the pipeline

The pipeline takes a source turtle program and a set of safety properties as input. Using Chiron's API, the program is translated to Chiron-IR and its relevant user and state variables are extracted. The program semantics are then encoded as Constrained Horn Clauses (CHCs), and combined with the properties expressed in CHC form. This CHC system is given to Z3's **SPACER** engine, which checks whether an **unsafe** state is reachable. A **sat** result yields a concrete counterexample, while **unsat** proves that the property holds.

Features

Category	Option	Description
Mode	universal	Checks the property over all possible values of the user input variables, with the turtle initially at the origin and pendown=True .
	default	Checks the property with all user variables initialized to 0, with the turtle initially at the origin and pendown=False .
	specific	Similar to default , except that the user may explicitly assign values to some or all user input variables.
Property Scope	all	Interprets the safety guarantee as required to hold at every point during program execution.
	terminating	Interprets the safety guarantee as required to hold only at program termination.

Table 1. Verification modes and property scopes supported by the verifier.

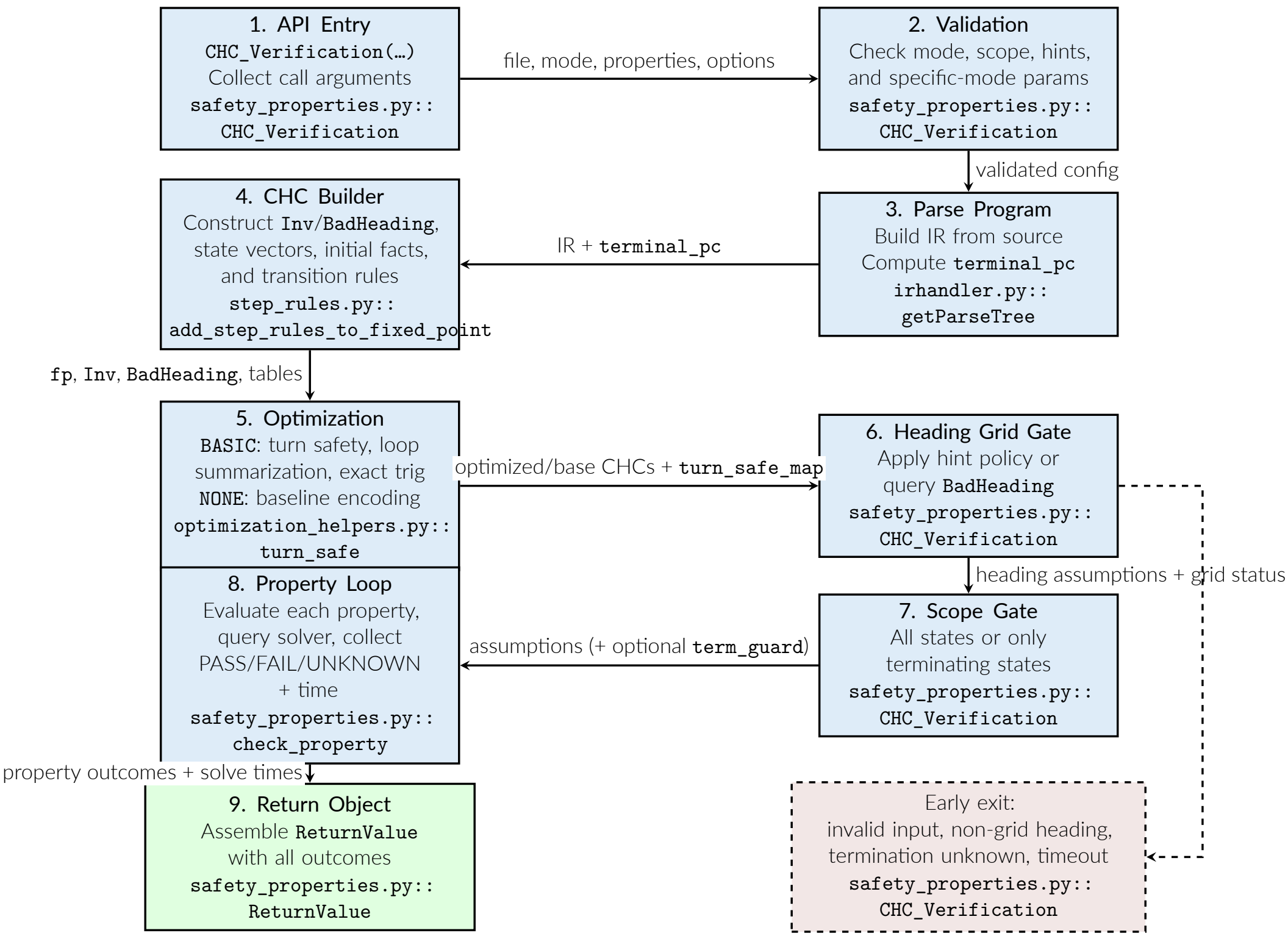


Figure 1. Main verification pipeline showing stage responsibilities and data artifacts flowing through parsing, CHC construction, heading/scope controls, and final result aggregation.

Outputs & Perf Analysis

Design Choices & Optimizations

Design Choices

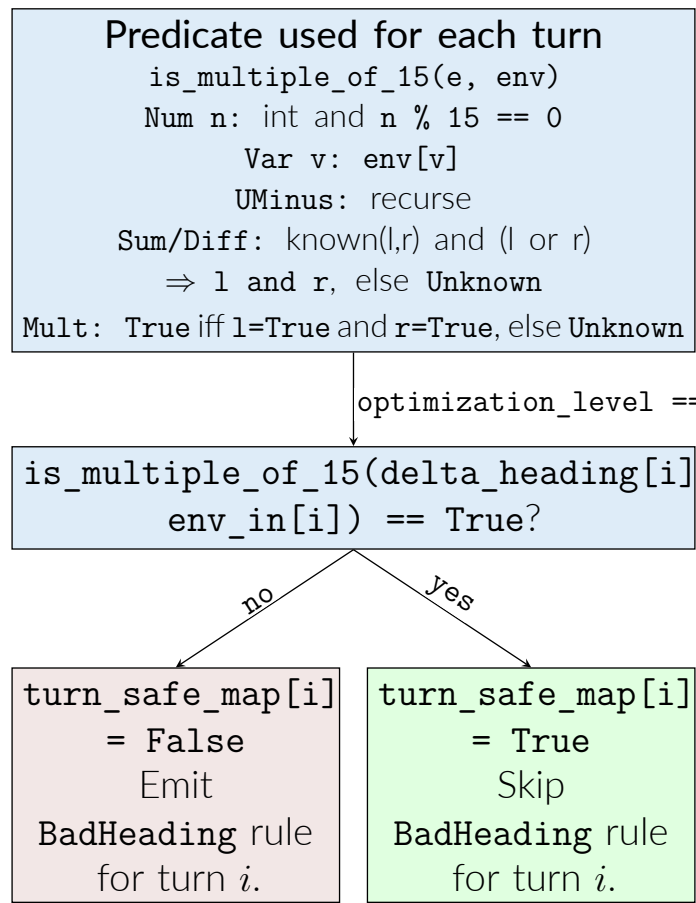
This section summarizes the principal modeling and solver choices adopted in the verifier.

- **CHC semantics with Z3 Fixedpoint/SPACER:** Program behavior is encoded as CHCs and solved using Z3 Fixedpoint with **engine='spacer'** (an IC3/PDR-style Horn-clause engine), with safety posed as **Exists(s, Inv(s) & Not(P(s)))**.
- **Single global invariant schema:** A unified relation **Inv(pc, xcor, ycor, heading, pendown, user_vars, loop_counters)** is used across initialization, transition rules, and property queries.
- **Heading-lattice geometry model:** Heading is normalized to $[0, 360)$ and constrained to $\{0, 15, \dots, 345\}$, with violations represented explicitly via **BadHeading**.

Design Choice	Benefit	Tradeoff
CHCs solved by Z3 SPACER	Supports unbounded safety check-Performance remains sensitive to ing through inductive invariant syn- nonlinear arithmetic obligations; thesis and returns counterexample solver outcomes may be UNKNOWN .	witnesses on violation.
Full state (pc+counters+vars)	Encodes control-state and data- Properties over a strict subset of vari- state dependencies in one rela- ables still incur full-arity reasoning, in- tion: pc for control/termination, creating solver cost.	xcor/ycor/heading for geometric predicates, and loop counters for repeat constructs.
Strict heading lattice + BadHeading	Avoids direct transcendental Lookup expansion must remain sin/cos constraints in SPACER bounded to control CHC size; failure by using a finite 15-degree trigono- to prove grid preservation yields metric lookup table (precomputed conservative UNKNOWN outcomes. in Python) with explicit BadHeading obligations.	

Optimizations

Turn-Safe Heading Inference



```
if :u > 0 [  
  :t = 30  
] else [  
  :t = 45  
]  
left :t  
forward 20
```

NONE **UNKNOWN** timeout (60 s)
BASIC **PASSED** solve 0.055 s

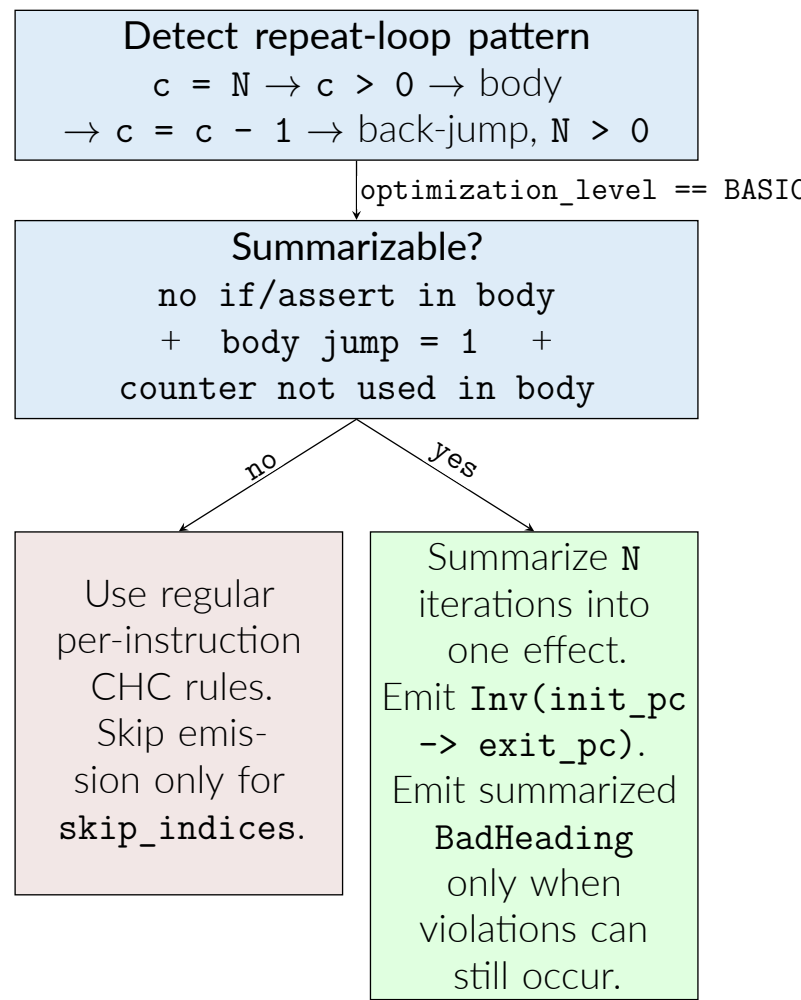
Why: **:t** can only be 30 or 45, so both branches stay on the 15-degree heading grid. The divisibility check (**n % 15 == 0**) is resolved statically on numeric literals, without adding extra arithmetic constraints to Z3. Under BASIC, this turn is marked safe and its **BadHeading** rule is not emitted. That shrinks the CHC system and avoids the heading-safety timeout seen in NONE.

Repeat-Loop Summarization

```
:b = 0  
repeat 100 [  
  :a = :a + 1  
  :b = :b + :a  
]
```

NONE **PASSED** solve 35.16 s
BASIC **PASSED** solve 0.063 s

Why: BASIC detects this repeat body as summarizable and composes all 100 iterations into one summarized loop effect. Instead of exploring each loop step through many CHC transitions, the solver reasons over a single compact relation from loop entry to exit. This drastically reduces search depth and clause churn during solving. The extra work shifts to build-time summarization, yielding a much faster proof query.



Future Work

- **Property-aware state projection:** Project the state space onto only the variables relevant to a given property, shrinking the CHC encoding and reducing solver load.
- **Stronger turn-safety check:** The current heading-grid check conservatively requires both operands of a product to be multiples of 15°. Incorporating richer program invariants will reduce unnecessary **UNKNOWN** verdicts.
- **Recursive loop summarization:** For perfectly nested affine loops, summarize the inner loop first into a single affine state transformer, then summarize the outer loop with it — drastically reducing CHC clause counts for deep nesting.
- **User-specified focus variables (focus_vars):** Let users name the variables most relevant to a property so the solver reasons over a